箕面ハイキングマップ デプロイ手順書

バージョン: 1.10 **最終更新日:** 2026年10月5日 **対象:** 運用・開発担当者 **関連:** 機能仕様 `funcspec-202611.md` / 公開API 仕様 `publish-api-202609.md`

---

1. 本書について

本アプリの**配信のしかた**と、変更の種類ごとに**何をすればユーザーに届くか**をまとめる。

配信先が2つ（Vercel / GitHub Pages）あり、**GitHub Pages で確認してから Vercel（本番）に出す** 2段階の運用をしている。さらに地図データは**デプロイを伴わない公開**で更新されるため、「どれを直したときに何が必要か」を取り違えやすい。判断はここに一本化する。

---

2. 配信先と役割

配信先	位置づけ	配信するもの	反映のきっかけ
GitHub Pages	**確認用**（先行公開）	アプリ（`public/`）のみ	`main` への push（`.github/workflows/pages.yml` が自動実行）
Vercel（Production）	**本番**	アプリ（`public/`）+ **公開API**（`api/`）+ 公開ストア（Vercel Blob）	**`release` ブランチへの push**（リポジトリ連携）／環境変数の変更後は再デプロイ
Vercel（Preview）	確認用（API）	同上（Blob は**本番と共用**）	`main` への push（コミットごとに Preview が作られる。本番は変わらない）

- **流れ:** `main` に push → **GitHub Pages で確認** → 確認したコミットを **`release` に push** → Vercel の本番（`https://minoh-hiking.vercel.app/`）に出る（手順は 5.1）。 `main` に push しただけでは、Vercel の本番は変わらない。
- 公開API は **Vercel にしか無い**。GitHub Pages 版アプリは `https://minoh-hiking.vercel.app/api/*` （**本番の API、つまり `release` の `api/`**）をクロスオリジンで参照する（`config.js` が `github.io` を判定して切り替える。API 側は CORS を全許可）。 そのため **`api/` の変更は GitHub Pages では確認できない**（→ 5.4）。
- したがって **API を止めると両方のアプリでデータが出なくなる。**
- GitHub Pages は確認用だが、**URL を知っていれば誰でも開ける。**確認前の変更を利用者が使うこともある 「先行公開版」と考えておく（GitHub Pages で開いた画面の QR コードは GitHub Pages の URL になり、 ホーム画面に追加した端末には確認前の更新も届く）。アプリ内の URL 表示・README・ポートフォリオは Vercel の本番を指している。

```
push (main)
  |
  +--> GitHub Pages     :  public/ (app) only          ... 確認用（先行公開）
  |                            |
  |                            +-- GET --> https://minoh-hiking.vercel.app/api/*
```

```
(本番 API)
  |
  +--> Vercel Preview   :  public/ + api/              ... API の確認用（Blob は本番と共用）

  (GitHub Pages で確認)

push (<確認したコミット>:release)
  |
  +--> Vercel Production:  public/ (app) + api/ (publish API) + Blob (store)
                           https://minoh-hiking.vercel.app/

MapPublisher -- POST /api/mapdata, /api/closures --> Blob -- GET --> 両方のアプリ
```

> Vercel の本番のブランチは、管理画面の **Settings → Environments → Production → Branch Tracking** で `release` に設定している。`main` に戻すと、`main` への push がそのまま本番に出る（GitHub Pages での確認を経なくなる）ため変えないこと。 `release` は `main` の後を追うだけのブランチで、**直接コミットしない**（→ 5.1 ⑩）。

---

3. 変更の種類とやること（早見表）

変更したもの	必要な作業	デプロイ
地図データ・通行止めの内容	MapPublisher で公開する	**不要**
オフライン地図のタイル範囲	DownloadArea で出力 → MapPublisher で公開する	**不要**
`public/` のコード・画像	`SHELL_CACHE` をバンプ → `main` に push → GitHub Pages で確認 → `release` に push（→ 5章）	要
`api/` のコード	`main` に push → **Vercel の Preview で確認** → `release` に push（→ 5.4）	要
Vercel の環境変数	値を設定 → **再デプロイ**（設定だけでは反映されない）	要
`docs/` のみ	`main` に push（`.md` を直したら `.pdf` も作り直す）。`release` への push は不要	影響なし

`public/shell-revisions.json`（シェルの内容ハッシュ一覧）は**デプロイ時に自動生成**される。 手で書き換えるファイルではなく、リポジトリにも置かない（Vercel は `vercel.json` の `buildCommand`、GitHub Pages は `.github/workflows/pages.yml` のステップで `scripts/gen-shell-revisions.mjs` を実行する）。

「データを直すたびにアプリを出し直す」必要は無い。 ポイント・ルート・スポット・通行止め・ タイル範囲は すべて公開API 配信であり、MapPublisher からの公開だけでユーザーに届く。

---

4. 事前設定（初回・環境を作り直したときのみ）

Vercel プロジェクト **minoh-hiking** に必要なのは、**Blob ストアの接続**と**公開トークン**の2つだけ。 どちらも**設定した後の再デプロイで初めて有効**になる。4.1 から順に、1回だけ実施する。

> 画面の項目名は Vercel 側の更新で変わることがある。表記が違うときは 「Storage（ストレージ）」「Environment Variables（環境変数）」「Redeploy（再デプロイ）」 に当たる場所を探す。

4.1 Blob ストアを接続する

公開データ（地図データ・通行止め）の実体を置く場所。**接続するだけでよい**。 フォルダやファイルを手で作る必要は無く、初回公開時に `api/_lib/store.js` が作る。

1. https://vercel.com/ にログインし、プロジェクト **minoh-hiking** を開く
2. 上部タブの **Storage** を開く
3. **Create Database**（すでにあるストアを使うときは **Connect Store**）→ **Blob** を選ぶ
4. ストア名を入れて作成する（例: `minoh-hiking-blob`。名前は任意。Region は既定のままでよい）
5. 接続先プロジェクトに **minoh-hiking** を選び、Environment は **Production / Preview / Development** すべてにチェックして **Connect**
6. **Settings** → **Environment Variables** に `BLOB_READ_WRITE_TOKEN` が 自動追加されたことを確認する（**値は開かない・コピーしない・Git に入れない**）

> **Preview・Development も本番と同じストアを使う。** 5 ですべての Environment に接続するため、`main` への push で作られる Preview の API や `vercel dev` も、**本番のデータを読み書きする** （確認用の別データは無い）。Preview での確認は読み取り（GET）に留める（→ 5.4）。

> **変数名は既定のままにする。** 接続時に Environment Variables のプレフィックスを付けて `〇〇_READ_WRITE_TOKEN` にすると、`@vercel/blob` が読むのは `BLOB_READ_WRITE_TOKEN` だけなので 公開時に `500` になる（`api/_lib/store.js` は token を渡していない）。
>
> ストアの Settings → Quickstart の **`.env.local` タブに出る `BLOB_STORE_ID` / `BLOB_READ_WRITE_TOKEN` は「ローカル開発用にコピーする値」の表示**で、そのままでよい。 Vercel 上は接続で設定済み。`BLOB_STORE_ID` は本アプリでは使わない（OIDC 認証用）。

接続後、公開のたびに Blob 上へ次のパスが作られる（定義は `api/_lib/datasets.js`）。

Blob 上のパス	中身
`manifest.json`	各データセットの version・updatedAt・件数（`GET /api/manifest` の実体、**採番の基準**）
`mapdata/minoh-hiking-mapdata.geojson`	緊急ポイント・ルート・スポット（最新）
`mapdata/previous.geojson`	同・前回分（1世代のみ）
`closures/minoh-hiking-closure.geojson`	通行止め・通行困難地点（最新）
`closures/previous.geojson`	同・前回分（1世代のみ）

> **フォルダを手で作る必要は無い。** Blob に「フォルダ」という実体は無く、パス名の `/` を 管理画面がフォルダのように見せているだけ。すでに運用している `closures/` はそのまま使い （本体のパスは旧方式から変えていない）、`mapdata/` と `manifest.json` は**初回公開のときにできる**。

> `previous.geojson` は退避元ができる2回目の公開から作られる。 旧方式の `closures/history/` は新方式では使わない（削除は 6.3）。

> ストアはプロジェクトではなく**アカウント（チーム）に属する**。作り直すと中身は空になり、`manifest.json` が無くなるため version の採番も 1 からやり直しになる。 中身を残したまま別プロジェクトから使いたいときは、作り直さず **Connect Store** で接続する。

4.2 公開トークン `MAP_PUBLISH_TOKEN` を設定する

MapPublisher からの公開（POST）を通すための共有トークン。mapdata・closures 共通で1つ。

1. ランダムな32バイトの値を作る（PowerShell）：

```
$b = [byte[]]::new(32)
[System.Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b)
[Convert]::ToBase64String($b)
```

2. **環境変数の画面を開く**（直接 URL が確実）

   ```
   https://vercel.com/<アカウント名またはチーム名>/minoh-hiking/settings/environment-variables
   ```

   画面からたどる場合は、**プロジェクト minoh-hiking を開く → 上部タブ Settings → 左メニュー Environment Variables**。 ここは**プロジェクトの Settings** で、Blob ストア側の Settings（Quickstart がある画面）には 環境変数の追加欄は無い。

3. **Add New**（版により **Create new** / ＋）を押して登録する。入力欄が最初から出ている版も ある。`.env` を貼り付ける欄しか見当たらないときは `MAP_PUBLISH_TOKEN=値` の形式で貼ってもよい

項目	値
Key	`MAP_PUBLISH_TOKEN`
Value	1 で作った文字列（**32文字以上**）
Environments	**Production**（必須）。Preview にも設定できるが、Preview の API も本番と同じ Blob を読み書きする（4.1）ため、**Preview への公開は本番のデータを書き換える**（試しの公開にはならない）
Sensitive	選べるならオン（登録後は値を読み出せなくなる）

4. **Save** する

5. 値はパスワードと同じ扱いにする。Git・`public/`・チャットに貼らない （`.gitignore` が `.env` / `.env.local` を除外しているのは、ローカルに置いたときの保険）

4.3 再デプロイして有効化する

環境変数は、すでに動いているデプロイには入らない。 設定したら必ず出し直す。

1. **Deployments** タブを開く
2. 最新の **Production** デプロイ（`release` のコミット。`main` の push で並ぶ Preview ではない）の 右端 ⋯ → **Redeploy**
3. ダイアログの **Redeploy** を押す（Build Cache の使用有無はどちらでもよい）
4. Status が **Ready** になるまで待つ

> GitHub Pages 側には API が無いため、この作業は不要（→ 2章）。

4.4 設定できたか確認する

```
# (a) トークン無しの POST。401 が正しい状態
curl.exe -s -o NUL -w "%{http_code}\n" -X POST -H "Content-Type: application/json"
-d "{}" https://minoh-hiking.vercel.app/api/mapdata

# (b) 配信状況。未公開なら version は空文字で返る
curl.exe -s https://minoh-hiking.vercel.app/api/manifest
```

結果	意味
(a) が `401`	`MAP_PUBLISH_TOKEN` が有効。正しい状態
(a) が `503`	トークン未設定、または**設定後に再デプロイしていない**（→ 4.2 / 4.3）
(b) が JSON を返す	API は動いている（**Blob 未接続でも空の JSON が返る**ため、これだけでは接続の確認にならない）

Blob 接続の最終確認は、MapPublisher から実際に1回公開すること。 `200`（`version` と `count` が返る）なら接続できている。`500`（公開ストアへの保存に失敗しました）なら Blob 未接続か、接続後に再デプロイしていない。

4.5 MapPublisher 側にトークンを登録する

公開操作を行う MapPublisher（別リポジトリの運用担当者用アプリ）に、4.2 と**同じ値**を保存する。

- Vercel 側だけ変えると公開が `401`、MapPublisher 側だけ変えても `401` になる
- **トークンを変えるときは、Vercel（+ 再デプロイ）と MapPublisher の両方を必ず揃える**

4.6 完了チェックリスト

- [ ] Storage に Blob ストアがあり、minoh-hiking に接続されている
- [ ] `BLOB_READ_WRITE_TOKEN` が Environment Variables にある（自動追加）
- [ ] `MAP_PUBLISH_TOKEN` を Production に設定した
- [ ] 設定後に Redeploy し、Ready になった
- [ ] トークン無しの POST が `401`（`503` ではない）
- [ ] MapPublisher に同じトークンを登録した
- [ ] 試しに1回公開して `200` が返った

> ローカルで `vercel dev` を使って API まで動かすときだけ、Vercel CLI で環境変数を取り込む（`vercel link` → `vercel env pull .env.local`）。`public/` を静的配信するだけなら不要。

5. 通常のデプロイ（アプリの更新）

`public/` または `api/` を変更したときの手順。 **データの内容だけを直したときは、この章は不要**（MapPublisher からの公開で届く → 3章）。

`main` に push して GitHub Pages で確認し（①〜⑨）、確認したコミットを `release` に push して Vercel の本番に出す（⑩〜⑪）という2段階で進める。本書で「**リリース**」と言うときは、 `release` への push（Vercel の本番への反映）を指す。

5.1 手順

確認用に出す（`main` への push → GitHub Pages）

① 変更内容とブランチを確認する

```
git status --short
git branch --show-current   # main であること
```

② `public/` を変更したなら `SHELL_CACHE` をバンプする（`public/service-worker.js`）

```
Select-String -Path public/service-worker.js -Pattern "SHELL_CACHE = "
# 例: const SHELL_CACHE = 'app-shell-2026-08-19.1';
```

- 命名は `app-shell-yyyy-mm-dd.n`。**日付は `main` にコミットする日**、`n` はその日の連番 （初回 `.1`、同じ日の2回目は `.2`、10回目以降は `.10`）。リリースする日に合わせる必要はない。 確認中に何度バンプしても、Vercel 版の利用者にはリリースしたときの名前が1回届くだけ （判定は名前が違うかどうかだけで、大小は比べない）
- ⚠ **忘れると、端末は旧 UI のまま更新されない。** 更新の確認（`SHELL_CACHE` 比較 → confirm）はキャッシュ名の違いで判定するため、 名前が同じだと新しいシェルを出しても端末は気づかない
- `api/` や `docs/` だけの変更ならバンプ不要

③ ファイルを追加したときは `SHELL_LOCAL_PATHS` にも足す（同じファイル）

- JS モジュール・アイコン・画像など、オフラインでも要るものはすべて
- 入れ忘れると、オフライン起動時にそのファイルだけ取得できない

④ 差し替えた旧ファイルは、このリリースでは消さない

- 端末には旧 `index.html` / 旧 `app.js` がキャッシュされたまま残ることがあり、 消すと 404 になって画像が出ない・モジュールが読めない、といった壊れ方をする
- 削除待ちの一覧は `service-worker.js` の `SHELL_LOCAL_PATHS` 直下の注記にまとめてある
- 全端末が更新を通したと判断できる次のリリース以降に消す。待つ期間は**差し替えを `release` に push した日から**数える（GitHub Pages に出た日ではない。利用者の多い Vercel 版に届くのはリリース時）

⑤ ローカルで表示を確認する

```
cd public; python -m http.server 8123    # → http://localhost:8123/
```

- `/api/*` は 404 になる（ローカルに API は無い）。地図データ・通行止めが出ないのは想定どおりで、「情報・言語/Info & Language」の「バージョン情報」の該当行と件数は `-` になる
- API 込みで見たいときだけ `vercel dev`

⑥ `docs/*.md` を直したときは `docs/*.pdf` も作り直す

⑦ コミットして `main` に push する

```
git add -A
git commit -m "変更内容の要約"
git push origin main
```

⑧ 確認用の配信が更新されたか見る

配信先	見る場所	正常
GitHub Pages	リポジトリの Actions → **Deploy to GitHub Pages**	緑（失敗なら `workflow_dispatch` で再実行）
Vercel（Preview）	Deployments タブ	当該コミットの **Preview** が **Ready**（本番はまだ変わらない。`api/` の確認に使う → 5.4）

⑨ GitHub Pages 版で 7.2 の動作確認を行う

- `https://sato4app.github.io/minoh-hiking/` を開く。更新の確認が出たら OK で最新にする
- 「情報・言語/Info & Language」→「バージョン情報」の**アプリバージョンが ② で付けた版**（キャッシュ名の `app-shell-` より後ろ。例: `2026-09-19.12`）になっていることを確かめる
- 直すところが見つかったら ② に戻る（`release` には出さない）

本番に出す（`release` への push → Vercel）

⑩ GitHub Pages で確認したコミットを `release` に push する

```
git fetch origin
git log --oneline -1 origin/main           # GitHub Pages に出ているコミット（確認したものと同じか見る）
git log --oneline origin/release..origin/main   # 今回のリリースに入るコミットの一覧
git push origin <確認したコミット>:release     # 例: git push origin 231cfaf:release
```

- **確認したコミットを指定して push する。** `git push origin main:release` は**手元の** `main` を 送るため、確認していないコミットが手元にあると一緒に本番に出る
- `release` は `main` を追いかけるだけ（fast-forward）。**`release` に直接コミットしない**

- push が non-fast-forward で断られたときは、`release` に `main` に無いコミットがある。 `--force` で上書きせず、`git log origin/main..origin/release` で中身を調べる

⑪ 本番が更新されたか見て、7章の確認を行う

配信先	見る場所	正常
Vercel（Production）	Deployments タブ	⑩ のコミットの **Production** が **Ready**

- `https://minoh-hiking.vercel.app/` で 7.2 を確認する（アプリバージョンが ⑨ と同じ版）
- `api/` を変えたリリースなら 7.1 も行う
- 本番に出ている版は、次のコマンドでも確かめられる

```
curl.exe -s https://minoh-hiking.vercel.app/service-worker.js | Select-String
"SHELL_CACHE = "
git ls-remote origin main release   # 2つが同じコミットなら、確認用と本番が揃っている
```

5.2 ユーザーへの反映

- **GitHub Pages 版の利用者には `main` への push で、Vercel 版（本番）の利用者には `release` への push で届く。** 以下は、それぞれの版に届いた後の端末での動き
- 起動時画面のボタン（どれでも）をタップしたときに「新しいバージョンのアプリが利用可能です」の 確認が出る（起動しただけでは出ない）
- OK を押すと最新を取得して再読み込みする。**ダウンロード済みの地図タイルは消えない**
- キャンセルした端末は、そのセッションでは再び確認しない。次に開き直してボタンをタップしたときに もう一度確認が出る
- 反映は**端末が次に起動し、起動時画面のボタンをタップしたとき**。全端末に行き渡るまで日数がかかる前提で、 旧ファイルの削除は次のリリース以降にする（→ ④）

5.3 起動画面の画像を差し替えるとき

起動画面の画像（`public/index.html` の `.home-image`）は WebP で配信し、 その**元データ（LibreOffice Draw の `.odg`）を `assets/` に置く**。

`public/` は `vercel.json` の `outputDirectory` に指定されており、置いたファイルは 全部そのまま配信される。配信の必要がない作業用ファイルは入れない。

① `.odg` を `assets/` に置く

Draw で編集したファイルをそのまま置く。PNG を経由する必要はない。 WebP は**非可逆**（`quality=90`）なので、サイズや品質を変えるときは配信中の WebP からではなく **必ず `.odg` から変換し直す**（WebP を種にすると劣化が重なる）。

② 変換する

```
python scripts/odg-to-webp.py assets/Startup-20260825.odg 1024
```

- 第2引数は**作品の幅**［px］。**1024 を既定**とする。実機は 900〜1030 デバイスpx で描画するため（iPhone 15 Pro Max で 1032、iPhone 15 で 943）、1024 あれば拡大されない
- 出力は `public/icons/Startup-<幅>x<高>.webp`。名前は解像度から自動で決まる
- Draw の**文字はベクター**なので、大きく書き出すほど鮮明になる。ただし背景に貼った画像は 埋め込み解像度が上限（`Startup-20260825.odg` は 768×1377）

> **`--convert-to` を直接使わないこと。** LibreOffice の書き出しは「ページ全体」を指定ピクセル数に 押し込むため、用紙が A4 のままだと ⑴ 用紙の縦横比と指定値の比が違えば画像が横に潰れ、⑵ 作品の周りに用紙の余白が白帯として入る。スクリプトは `.odg` から用紙寸法と作品の外接矩形を 読み、作品が目標幅になるページピクセル数を逆算してから切り抜くことでこれを避けている。

③ 切れていないか確かめる

スクリプトは切り抜き枠のすぐ外側に描画が残っていないかを調べ、`はみ出し: なし` と出す。`⚠ はみ出し: 下 側に描画があります` と出たときは、文字が図形の枠を越えて描かれている（`.odg` の図形サイズより実際の描画が大きい）。`--grow` を付けると用紙の地色に届くまで 枠を広げて切れを防ぐが、そのぶん余白が増える。**Draw 側で収めるほうが望ましい**。

④ 参照を更新する

解像度が変わるとファイル名も変わるので、次の3つを忘れないこと。

更新するもの	場所
`.home-image` の `src`	`public/index.html`
`SHELL_LOCAL_PATHS`	`public/service-worker.js`（→ 5.1 ③）
`SHELL_CACHE`	同上（→ 5.1 ②）

差し替え前の WebP は `public/` から**すぐに消さない**（→ 5.1 ④）。

> 透過について: 現行の素材は全画素が不透明なため、スクリプトは RGB（アルファ無し）で保存している。 透過が要る画像に差し替えるときは `scripts/odg-to-webp.py` の `convert('RGB')` を外す。

5.4 `api/` を変更するとき

GitHub Pages 版アプリは**いつも Vercel の本番の API（= `release` の `api/`）**を使う（→ 2章）。 そのため `api/` の変更は、`release` に出すまで GitHub Pages 版からは見えない。

① API は Vercel の Preview で確認する

- `main` に push すると、そのコミットの **Preview** が Vercel にできる（Deployments タブ → 当該コミットの Preview → **Visit** で URL が分かる）。Preview の `/api/*` はそのコミットの `api/` で動く
- Preview に保護（Vercel Authentication など）が掛かっているときは、Vercel にログインしたブラウザで開く
- **⚠ Preview の API も本番と同じ Blob を読み書きする**（→ 4.1）。 確認は**読み取り（GET）に留める**。Preview に `MAP_PUBLISH_TOKEN` を設定している場合、 Preview への公開（POST）は本番のデータを書き換える

② アプリの変更が新しい API に頼るときは、出す順番を考える

GitHub Pages で確認している間、新しいアプリは**古い（本番の）API** に対して動く。 新しい API が無いと動かない変更だと、GitHub Pages では正しく確認できない。次のどちらかにする。

- **アプリを、古い API でも動くように作る**（新しい項目が無ければ従来どおりに振る舞う、など）
- **API を先にリリースする。** `api/` だけのコミットを `main` に push → Preview で確認 → `release` に push した後で、アプリ側のコミットを `main` に push して GitHub Pages で確認する （2026.12 の移行と同じ考え方 → 6章）

API の契約（エンドポイント・応答・公開スキーマ）を変えるときは、 `publish-api-202609.md` の契約バージョンを更新し、MapPublisher 側にも反映する。

---

6. 今回の移行デプロイ（2026.12・1回限り）

地図データをアプリ同梱から公開API 配信へ切り替えるリリース。 **API とアプリを同時に出してはいけない。**

6.1 なぜ順序が要るか

アプリ側を先に切り替えると、初回公開が済むまでの間 `GET /api/mapdata` が空を返し、 **全ユーザーの地図からポイント・ルート・スポットが消える。**

`main` への push は Vercel と GitHub Pages の両方を同時に更新するため、 **2つのコミットに分け、間隔を空けて push する。**

> 上記は移行当時の運用（`main` への push が Vercel の本番にも出ていた）での記述。 現在は `release` への push で本番に出る（→ 2章）。API を先に出す必要がある リリースでは、API のコミットを先に `release` に出してからアプリ側を進める（→ 5.4）。

6.2 手順

#	作業	対象	確認
0	Vercel に `MAP_PUBLISH_TOKEN` を設定	Vercel	4章
1	**コミットA**（`api/` + `docs/` + `README.md`）を push	リポジトリ	`public/` は据え置き。ユーザーは同梱データのまま動き続ける
2	MapPublisher で **mapdata を初回公開**	MapPublisher	`2026.1` が採番される
3	配信内容を確認	curl	7.1
4	**コミットB**（`public/`）を push	リポジトリ	ここで初めてアプリが配信データを使う
5	表示を確認	ブラウザ	7.2

> 手順1の時点で **closures の公開トークンは `MAP_PUBLISH_TOKEN` に変わる**（旧 `CLOSURES_PUBLISH_TOKEN` は使われない）。手順0 を飛ばすと通行止めの公開が `503` になる。

6.3 リリース後の後始末

旧シェル（移行前の `index.html` / `app.js`）をキャッシュしたままの端末が残っている間は、 それらが参照するファイルを消すと影響が出る。**参照の仕方によって影響の重さが違う**ため、 下表の区分に従って実施す

る。

実施済み

#	作業	完了日
1	`public/data/minoh-emergency-points.geojson` / `public/data/minoh-hiking-routes-spots.geojson` を削除	2026-08-20
2	Vercel の環境変数 `CLOSURES_PUBLISH_TOKEN` を削除	2026-08-20
3	Blob 上の旧履歴 `closures/history/` を一括削除（前回分1世代の方式に移行済みのため）	2026-08-20
7	`vercel.json` の `/data/(.*)` のキャッシュ設定を削除（`public/data/` を廃止したため）	2026-08-23
8	`public/data/tile_manifest.json` を削除（タイル一覧は公開API 配信へ移行済み）	2026-08-23
9	`public/data/tile_buffers.geojson` を削除（どのシェルからも参照されていなかった）	2026-08-23

未実施

#	作業	旧シェルからの参照	消したときの影響	待つ必要
4	`public/closures.js` を削除	旧 `app.js` の **`import` 文**	**アプリが起動しない**（モジュール読み込み失敗）	**あり**
5	`public/icons/Startup-512x918.png` を削除	旧 `index.html` の `<img src>`	起動画面の画像が出ないだけ。**表示は継続**	小
6	`public/service-worker.js` のコメント（4・5 を「消さないこと」と記した注記）を削除	—	なし	4・5 と同時

4 が唯一の要注意項目である。 `import` は 404 でモジュールグラフ全体の読み込みが失敗するため、 アプリが起動しなくなる。5 と 1 は `<img src>` / `fetch()` であり、404 でも表示は続く。

「全端末が更新を通した」の判断

テレメトリは持たないため、期間で判断するほかない。 期間は、**その変更を `release` に push した日（Vercel の本番に出た日）から**数える。 GitHub Pages に出た日からではない（→ 5.1 ④）。 **端末がオンラインでアプリを開き、起動時画面のボタンをタップすれば**、`service-worker.js` を 読んで `SHELL_CACHE` を比較し、更新確認（confirm）が出る。取り残されるのは、長期間まったく 開かなかった端末と、更新確認をキャンセルし続けている端末。いずれもブラウザのキャッシュを 消せば復旧できる。 （2026.39 までのアプリ

には「起動時にアプリの更新版を確認」の設定があり、OFF の端末は 「バージョン情報等」を開くまで確認が出なかった。その端末もこの版に更新された後は、 ボタンのタップで確認が出る。）

---

7. 動作確認

7.1 API 単体（curl）

Windows PowerShell では `curl` が別のコマンドの別名になっているため、**`curl.exe`** と書く。

以下は**本番の API**（`release` の `api/`）に対する確認。`main` に push しただけでは変わらない。 リリース前の `api/` を確かめるときは、URL を Vercel の Preview のものに置き換え、GET だけを行う（→ 5.4）。

```
# version と件数（数百バイト）
curl.exe -s https://minoh-hiking.vercel.app/api/manifest

# 本体（件数だけ数える）
curl.exe -s https://minoh-hiking.vercel.app/api/mapdata  | node -e "let
s='';process.stdin.on('data',d=>s+=d).on('end',
()=>console.log(JSON.parse(s).features.length))"
curl.exe -s https://minoh-hiking.vercel.app/api/closures | node -e "let
s='';process.stdin.on('data',d=>s+=d).on('end',
()=>console.log(JSON.parse(s).features.length))"

# 認証: トークン無しの POST は 401 になること（401 以外なら公開口が無防備）
curl.exe -s -o NUL -w "%{http_code}\n" -X POST -H "Content-Type: application/json"
-d "{}" https://minoh-hiking.vercel.app/api/mapdata
```

確認すること:

- `GET /api/manifest` が `mapdata` / `closures` の version・updatedAt・件数を返す
- `GET /api/mapdata` の件数が公開した件数と一致する（初回公開なら 668 件）
- 公開のたびに version の連番が1つ進む（`2026.1` → `2026.2`）
- トークン無し・誤トークンの POST が `401`

7.2 アプリ側（ブラウザ）

GitHub Pages 版（`main` に push した後。5.1 ⑨）と **Vercel 版**（`release` に push した後。 5.1 ⑪）の**両方**で確認する。

- 「ハイキングマップ表示」で緊急ポイント・ルート・スポット・通行止めが描画される
- ルート線が端点（開始・終了ポイント）まで伸びている
- 「情報・言語/Info & Language」の「バージョン情報」を開くと、地図データ・通行止めの version と件数が出る（`-` でない）
- 2回目に開いたとき、DevTools の Network に **`/api/mapdata` が出ない**（version が一致するので本体を取りに行かない）
- 公開した直後に開き直すと、新しい version と件数に変わる
- オフライン（機内モード）で開いても、前回取得した内容が表示される

---

8. 復旧・ロールバック

状況	対処
誤ったデータを公開した	**正しいデータをもう一度公開する**（全置換）。直前の内容は Blob の `previous.geojson` に1世代だけ残っている
公開が `500` で失敗した	**もう一度公開する。** `manifest.json` が進んでいないため同じ version が採番される（冪等）。二重に version が飛ぶことはない
GitHub Pages で不具合を見つけた（リリース前）	`main` で直す（`git revert` か修正）→ GitHub Pages で確認し直す。**`release` には出さない**。本番は変わっていないので利用者への影響は GitHub Pages 版だけ
本番（Vercel）に不具合を出してしまった	Vercel の Instant Rollback で前のデプロイに戻す（すぐ効く）。**あわせて `main` を `git revert` →** GitHub Pages で確認 **→ `release` に push** する（`release` を戻さないと、次のリリースで再発する）。`release` を `--force` で巻き戻さない（`main` と食い違い、次の push が断られる）
GitHub Pages 側だけ古い	Actions の `Deploy to GitHub Pages` が失敗していないか確認し、`workflow_dispatch` で再実行する
Vercel の本番だけ古い（GitHub Pages は新しい）	`release` に push していない（`main` への push は Preview になるだけ）。`git ls-remote origin main release` で差を確かめ、GitHub Pages で確認済みのコミットを `release` に push する（→ 5.1 ⑩）

> ロールバックしても、端末のキャッシュは `SHELL_CACHE` の名前で判断される。 戻した版のキャッシュ名が新しい版と同じだと更新が検知されないため、 **切り戻し版でもキャッシュ名を1つ進めて出し直す**のが確実。

---

9. トラブルシューティング

症状	主な原因	対処
公開が `503`（トークン未設定）	`MAP_PUBLISH_TOKEN` 未設定、または**設定後に再デプロイしていない**	環境変数を確認して再デプロイ
公開が `401`	トークン不一致	MapPublisher の保存済みトークンを入れ直す
公開が `400`	データが検証に通らない	API が返す日本語メッセージのとおりに直す（判定は `publish-api-202609.md` §6 にのみ置いている）
公開したのにアプリに出ない	端末がまだ更新版を確認していない	起動時画面に戻ってボタンのどれかをタップする（またはアプリを開き直す）。公開は各端末が次に起動したとき、または起動時画面のボタンをタップしたときに反映される。マップ画面を開いたままの間は確認しない

症状	主な原因	対処
旧 UI と新 UI が混ざる	`SHELL_CACHE` のバンプ漏れ	キャッシュ名を進めて出し直す
端末で地図が出ない・一部の操作が効かない（コンソールに `Cannot read properties of null`）	端末に**古い** `index.html` が残ったまま新しい JS が読み込まれた（シェルの新旧が混ざった状態）	起動時画面のボタンをタップ → 更新確認で OK。直らなければブラウザのサイトデータを削除する。アプリ 2026.57 以降は初期化が途中で止まらず、地図表示と更新確認は動く
起動画面の画像やモジュールが 404	差し替えた旧ファイルを同じリリースで消した	ファイルを戻し、次のリリース以降に削除する
オフラインで地図データが出ない	Service Worker の掃除で `mapdata-cache` / `closures-cache` を消している	`service-worker.js` の `APP_MANAGED_CACHES` に入っているか確認する
GitHub Pages 版だけデータが出ない	Vercel 側の API が落ちている／CORS 設定の変更	`curl.exe` で Vercel の API を直接確認する
push したのに Vercel 版が変わらない	`main` にしか push していない（Vercel の本番は `release` を見ている）	確認後に `release` へ push する（→ 5.1 ⑩）
GitHub Pages 版で、新しい API を使う機能だけ動かない	`api/` の変更がまだ本番（`release`）に出ていない。GitHub Pages 版は本番の API を使う	API を先にリリースする（→ 5.4）
`release` への push が non-fast-forward で断られる	`release` に `main` に無いコミットがある	`--force` で上書きせず、`git log origin/main..origin/release` で中身を調べる

10. デプロイ前チェックリスト

`main` に push する前

- [ ] `public/` を変更した → `SHELL_CACHE` をバンプした
- [ ] JS ファイルを追加した → `SHELL_LOCAL_PATHS` に追加した
- [ ] 差し替えた旧ファイルを**このリリースでは消していない**（消すのは、前回それを `release` に出してから十分に期間が経ったとき）
- [ ] `api/` を変更した → アプリが古い API でも動くか、API を先にリリースする段取りになっている（→ 5.4）
- [ ] 環境変数を変えた → 再デプロイする段取りになっている
- [ ] `docs/*.md` を直した → `docs/*.pdf` を作り直した
- [ ] データの内容だけの変更なら、**デプロイではなく MapPublisher からの公開**で足りると確認した

`release` に push する前

- [ ] GitHub Pages 版で 7.2 の確認をした（アプリバージョンが今回の `SHELL_CACHE`）
- [ ] `api/` を変更した → Vercel の Preview で GET の確認をした
- [ ] push するのは **GitHub Pages で確認したコミット**（`git log --oneline -1 origin/main` と一致）
- [ ] `git log --oneline origin/release..<確認したコミット>` で、今回本番に出るコミットを見た

---

11. 変更履歴

版	日付	内容
1.0	2026-08-18	初版。配信先2つの役割、変更の種類ごとの作業、2026.12 の移行デプロイ手順（API とアプリを分けて出す）、確認・復旧・トラブルシューティングをまとめた
1.1	2026-08-19	4章（事前設定）と5章（通常のデプロイ）を画面操作・コマンドのレベルまで具体化。Blob ストアの接続手順・Blob 上のパス一覧・トークンの生成と登録・再デプロイ・設定確認（401/503 の読み分け）と、デプロイ手順の各ステップにコマンドと確認箇所を追記
1.2	2026-08-20	アプリシェルの更新方式の変更に追随。デプロイ時に `shell-revisions.json` を自動生成する旨を3章に追記し、5.2・6.3 の「裏で自然に最新化される」記述を実装に合わせて修正（更新確認 → OK が唯一の更新経路。「起動時にアプリの更新版を確認」が OFF の端末は自動更新されない）
1.3	2026-08-23	`public/data/` の廃止に追随。後始末の 7・8・9（`vercel.json` の `/data/(.*)` ルール削除、`tile_manifest.json` / `tile_buffers.geojson` の削除）を実施済みへ移し、7 の解説を削除した
1.4	2026-08-25	5.3「起動画面の画像を差し替えるとき」を追加。元画像（PNG）は配信対象の `public/` ではなく `assets/` に置くこと、WebP は非可逆のため必ずマスターから変換し直すこと、解像度がファイル名に入るため差し替え時は参照と `SHELL_CACHE` の更新が要ることを明記した
1.5	2026-08-25	5.3 を、元データを PNG から LibreOffice Draw の `.odg` に改めた手順へ全面的に書き直した。`scripts/odg-to-webp.py`（用紙寸法と作品の外接矩形から逆算して書き出し・切り抜く）の使い方、書き出し幅 1024px の根拠、切れの確認方法を追記
1.6	2026-09-19	アプリ 2026.40 に追随。アプリの更新確認が「起動時（設定 ON のとき）／「バージョン情報等」を開いたとき」から「起動時画面のボタンをタップしたとき」に変わり、「起動時にアプリの更新版を確認」の設定が無くなったため、5.2 と 6章「全端末が更新を通した」の判断の説明を改めた。「バージョン情報」がホームのボタンから「設定/Settings」内のトグルへ移ったため、5.1・7.2 の確認箇所の書き方を合わせた。あわせて 2026.41 の改称（「設定/Settings」→「設定・情報/Settings & Info」）に合わせた
1.7	2026-09-19	**Vercel の本番を `release` ブランチから出す運用に合わせて全体を改めた**（Vercel の Production の Branch Tracking を `release` にし、`main` への push で GitHub Pages に出して確認した後、確認したコミットを `release` に push して本番に出す）。本書は「`main` への push で Vercel の本番にも出る」前提のままだった。2章の表・図・流れ（GitHub Pages は確認用＝誰でも開ける先行公開版、`main` の push で作られる Vercel の Preview、Branch Tracking の設定場所、`release` に直接コミットしない）、3章の早見表、5.1 を「確認用に出す（①〜⑨）」と「本番に出す（⑩〜⑪）」の2段に分け、確認したコ

版	日付	内容
		ミットを指定して `release` に push する手順と確認方法を追加。`SHELL_CACHE` の日付は `main` にコミットする日とした。5.2 に版ごとの届くきっかけを追記。**5.4「`api/` を変更するとき」を新設**（GitHub Pages 版はいつも本番の API を使うため `api/` は Preview で確認する、Preview の API も本番と同じ Blob を読み書きするので GET に留める、新しい API に頼るアプリの変更は API を先にリリースする）。4.1・4.2 に Preview も本番のストアを使う注意、4.3 に再デプロイするのは Production であることを追記。旧ファイル削除の待ち期間（5.1 ④・6.3）は `release` に push した日から数えるとした。6.1 に移行当時の運用である旨を注記。7章・8章・9章・10章を2段階の運用に合わせて改め、8章に「GitHub Pages で不具合を見つけた」「Vercel の本番だけ古い」、9章に3件を追加。あわせて 4.1 の 6.3 へのリンク切れ（見出し名の変更に追随していなかった）を直した
1.8	2026-09-23	アプリ 2026.57 に追随。**9章に「端末で地図が出ない・一部の操作が効かない」**（古い `index.html` と新しい JS が混ざった状態）の行を追加した。初期化が途中で止まらなくなり、その状態でも地図表示と起動時画面のボタンからの更新確認は動く
1.9	2026-09-30	アプリ 2026.70 に追随。公開データ（地図データ・通行止め・タイル一覧）の更新版の確認が「起動時のみ」から「起動時と、起動時画面のボタンをタップしたとき」に変わったため、9章の「公開したのにアプリに出ない」の原因と対処を改めた
1.10	2026-10-05	アプリ 2026.72 に追随。起動時画面の「設定・情報/Settings & Info」が「情報・言語/Info & Language」に改称されたため、5.1・7.2 の確認箇所の書き方を合わせた